



TRABAJO FIN DE MÁSTER

Comparación de métodos clásicos y profundos para la detección de música generada por inteligencia artificial

Realizado por
Sergio Álvarez Piñón

Para la obtención del título de
**Máster en Ingeniería del Software: Cloud, Datos y Gestión
TI**

Dirigido por
José María Luna Romera

Convocatoria de Septiembre, curso 2025/26

Agradecimientos

Quiero agradecer a X por...

También quiero agradecer a Y por...

Resumen

Incluya aquí un resumen de los aspectos generales de su trabajo, en español.

Palabras clave: Palabra clave 1, palabra clave 2, ..., palabra clave N

Abstract

This section should contain an English version of the Spanish abstract.

Keywords: Keyword 1, keyword 2, ..., keyword N

Índice general

1. Introducción	1
1.1. Introducción	1
1.2. Estructura de este documento	1
2. Estado del arte	3
2.1. Introducción y delimitación del problema	3
2.2. Conjuntos de datos para detección de música sintética	3
2.3. Enfoques clásicos con características acústicas	4
2.4. Enfoques profundos y representaciones preentrenadas	5
2.5. Segmentación y evaluación a nivel de canción	5
2.6. Generalización, robustez y limitaciones actuales	6
2.7. Conclusiones	6
3. Estudio previo	8
3.1. Introducción	8
3.2. Objetivos	8
3.2.1. Objetivo general	8
3.2.2. Objetivos específicos	8
3.3. Metodología	9
3.3.1. Documentación y revisión bibliográfica	9
3.3.2. Gestión del proyecto y control de versiones	9
3.3.3. Preparación de los datos y reproducibilidad	9
3.3.4. Desarrollo y evaluación experimental	9
3.4. Planificación	10
3.5. Presupuesto	13
4. Descripción de la propuesta	16
4.1. Introducción	16
4.2. Formulación del problema	16
4.3. Selección del conjunto de datos AIME	16
4.4. Auditoría y construcción del subconjunto experimental	17
4.5. Diseño de las particiones	17
4.6. Baseline clásico MFCC + SVM	18
4.7. Modelo profundo preentrenado MERT	19
4.8. Medidas de evaluación	19
4.9. Limitaciones del diseño experimental	20
5. Validación	22
5.1. Introducción	22
5.2. Baseline MFCC + SVM	22
5.3. MERT congelado + SVM	23
5.4. Comparación y selección del modelo	25

6. Aplicación web VeriSon	27
6.1. Objetivo y alcance	27
6.2. Arquitectura y tecnologías	27
6.3. Backend y flujo de inferencia	28
6.3.1. API y validación de entrada	28
6.3.2. Preprocesamiento e inferencia	29
6.3.3. Agregación y respuesta	29
6.4. Frontend e interacción con el usuario	30
6.4.1. Flujo de interacción	30
6.4.2. Comunicación con el backend	31
6.4.3. Presentación del resultado	31
6.5. Despliegue e integración continua	33
6.5.1. Despliegue	33
6.5.2. Integración continua	33
7. Conclusiones	35
8. Bibliografía	36

Índice de figuras

3.1. Planificación temporal del TFM entre el 22 de junio y el 15 de septiembre de 2026.	12
5.1. Matriz de confusión del baseline MFCC + SVM en <i>test</i>	23
5.2. Matriz de confusión de MERT + SVM lineal en <i>test</i>	24
6.1. Arquitectura general de la aplicación web VeriSon.	27
6.2. Estado de VeriSon tras seleccionar un archivo WAV válido.	30
6.3. Resultado de un análisis con un archivo WAV.	31
6.4. Resultado de un análisis con un archivo MP3.	32
6.5. Mensaje mostrado tras rechazar un audio que supera la duración máxima permitida.	32
6.6. Checks obligatorios de integración continua superados en una <i>pull request</i>	34

1. Introducción

1.1. Introducción

La generación musical mediante inteligencia artificial ha avanzado de forma notable en los últimos años. Los modelos actuales permiten producir piezas musicales completas a partir de descripciones textuales u otras condiciones de entrada, y su uso se ha extendido más allá de entornos técnicos especializados. Este escenario abre posibilidades creativas y productivas, pero también dificulta distinguir con claridad entre música producida por humanos y música generada mediante IA.

Para estudiar esta distinción, el problema se formula como una tarea de clasificación binaria y se evalúan dos enfoques: uno clásico basado en *Mel-frequency cepstral coefficients* (MFCC) y *support vector machine* (SVM), y otro profundo basado en el modelo *Acoustic Music Understanding Model with Large-Scale Self-supervised Training* (MERT) y una SVM. El primero resume la señal mediante características acústicas compactas, mientras que el segundo utiliza representaciones profundas preentrenadas del audio.

Ambos enfoques se evalúan con los mismos ejemplos, etiquetas y particiones. El preprocesamiento se adapta a los requisitos de cada modelo, pero se mantiene dentro de unas condiciones comunes de evaluación que permiten analizar tanto el rendimiento predictivo como las medidas operacionales registradas.

La principal aportación del TFM consiste en definir un protocolo experimental común y reproducible sobre AIME para comparar un enfoque clásico basado en MFCC y SVM con otro profundo basado en MERT congelado y SVM. La evidencia predictiva y operacional obtenida se utiliza para seleccionar el modelo integrado en una prueba de concepto web, manteniendo el alcance de los resultados dentro del escenario experimental evaluado.

1.2. Estructura de este documento

El presente documento se organiza en los siguientes capítulos:

- **Introducción:** presenta el contexto del trabajo, la motivación del problema, el alcance de la clasificación binaria y la estructura general de la memoria.
- **Estado del arte:** revisa trabajos y recursos relacionados con la generación musical mediante IA, la detección de música sintética, los conjuntos de datos disponibles y los enfoques de análisis de audio relevantes para este TFM.
- **Estudio previo:** define los objetivos del TFM, la metodología general y los elementos de planificación y presupuesto asociados al desarrollo del proyecto.

- **Descripción de la propuesta:** describe el planteamiento experimental, incluyendo el conjunto de datos, el preprocesamiento, el baseline MFCC + SVM, el enfoque profundo basado en MERT y las condiciones comunes de evaluación.
- **Validación:** recoge la ejecución de ambos enfoques, las medidas de rendimiento predictivo y operacional, la comparación entre MFCC + SVM y MERT + SVM, y la selección del modelo para la prueba de concepto web.
- **Aplicación web VeriSon:** presenta la integración del modelo seleccionado en la prueba de concepto web y su arquitectura software.
- **Conclusiones:** sintetiza los resultados, las aportaciones y las principales limitaciones del trabajo.

2. Estado del arte

2.1. Introducción y delimitación del problema

La detección de música generada mediante inteligencia artificial se ha ido consolidando como un problema propio dentro del análisis de audio. En trabajos recientes, esta tarea aparece bajo la denominación *Synthetic Song Detection* (SSD), entendida como la clasificación entre canciones producidas por humanos y canciones generadas de extremo a extremo mediante sistemas de IA [18].

Esta delimitación es importante porque existen tareas cercanas que conviene distinguir. La *Singing Voice Deepfake Detection* se centra en detectar voces cantadas sintéticas o convertidas, a menudo dentro de mezclas donde el acompañamiento puede proceder de música real [20, 19]. La detección de canciones sintéticas de extremo a extremo, en cambio, considera que pueden ser artificiales la voz, la instrumentación, la letra, el arreglo y la producción global. SSD también se diferencia de la detección de voz hablada sintética, la detección de audio sintético general, la identificación de letras generadas mediante IA y la atribución del modelo generador.

El interés de SSD va más allá de obtener una decisión automática sobre un archivo de audio. La música es una señal con estructura temporal, patrones tímbricos, organización rítmica y relaciones de larga duración. Por ello, los detectores pueden apoyarse tanto en indicios locales de la señal como en propiedades musicales más amplias. Esta combinación explica que el campo incluya desde enfoques clásicos basados en características acústicas hasta modelos profundos que intentan aprovechar representaciones aprendidas y relaciones entre fragmentos.

2.2. Conjuntos de datos para detección de música sintética

En el contexto de SSD, SONICS aporta un conjunto de datos específico para detectar canciones generadas de extremo a extremo. Sus autores señalan que muchos corpus previos se centraban en voz cantada sintética, mientras que los sistemas actuales pueden generar también la instrumentación, el arreglo y la producción global [18]. Esta diferencia motiva la creación de conjuntos específicos para canciones completas y de protocolos capaces de considerar relaciones temporales más amplias.

FakeMusicCaps aborda un problema próximo desde otra perspectiva. El conjunto de datos se construye a partir de MusicCaps, regenerando pares música-texto mediante varios modelos *text-to-music*. Además de la detección binaria, el trabajo estudia la atribución del modelo generador [3]. FakeMusicCaps constituye un ejemplo de corpus específico para música generada mediante IA y muestra cómo la detección puede verse condicionada por los generadores incluidos en el conjunto de datos.

AIME ocupa una posición distinta. La tarjeta oficial del conjunto de datos lo presenta como *AI Music Evaluation Dataset*, con música generada por distintos modelos y ejemplos procedentes de MTG-Jamendo, junto con metadatos como el modelo de origen y una descripción basada en etiquetas [5]. Su finalidad original está vinculada a la evaluación de modelos de generación musical y preferencias humanas, por lo que su utilización en detección requiere definir y documentar un protocolo experimental propio [8].

La reutilización de AIME para detección cuenta al menos con un antecedente verificado: *Fusion Segment Transformer*, identificado como preprint de 2026, evalúa su propuesta en SONICS y AIME [12]. Esta evidencia es todavía limitada, aunque muestra que el corpus puede emplearse en trabajos de detección cuando se explicita el protocolo experimental.

2.3. Enfoques clásicos con características acústicas

Los enfoques clásicos de clasificación de audio suelen partir de características acústicas calculadas manualmente. Estas representaciones resumen propiedades de la señal antes de aplicar un clasificador, lo que permite construir sistemas relativamente simples, reproducibles y de bajo coste computacional. En este contexto, los MFCC representan información espectral de corto plazo mediante vectores compactos.

Los MFCC se han utilizado en tareas relacionadas de detección de audio sintético, especialmente en sistemas de anti-*spoofing* de voz. El precedente aplicado citado en este capítulo es Novoselov et al., que emplean MFCC dentro del reto ASVspoof 2015 y comparan clasificadores clásicos con enfoques más complejos [16]. Aunque este dominio no coincide con SSD, sirve como antecedente metodológico para entender su uso como representación acústica compacta en problemas de detección.

La SVM es un clasificador clásico para tareas supervisadas, especialmente adecuado cuando se trabaja con vectores de características fijas. Su formulación de margen máximo y el uso de kernels permiten construir fronteras de decisión no lineales sin requerir arquitecturas profundas [4]. La combinación MFCC + SVM constituye un baseline clásico reproducible y controlable, basado en características acústicas compactas.

La principal limitación de este tipo de aproximación es que resume la señal en descriptores compactos. Esto puede ser suficiente para capturar ciertos patrones espectrales, pero deja menos margen para modelar relaciones temporales largas, estructura musical o dependencias entre secciones de una canción. Estas limitaciones han favorecido el estudio de representaciones profundas capaces de modelar información acústica y temporal más compleja.

2.4. Enfoques profundos y representaciones preentrenadas

El desarrollo reciente de SSD se ha desplazado hacia modelos profundos capaces de aprender representaciones más ricas a partir del audio. SONICS propone SpecTT-Tra, una arquitectura orientada a capturar dependencias temporales largas con menor coste que algunos modelos convolucionales o transformadores de referencia [18]. Esta línea pone de manifiesto que la detección de canciones completas depende tanto de indicios locales como de la forma en que la música se organiza a lo largo del tiempo.

Otra línea dentro de los enfoques profundos consiste en reutilizar conocimiento aprendido previamente, en lugar de entrenar toda la arquitectura desde cero para el problema concreto. Los encoders de audio pueden entrenarse sobre corpus amplios y después emplearse como extractores de vectores para tareas de clasificación.

Dentro de esta línea se sitúa MERT, entrenado mediante aprendizaje autosupervisado a gran escala sobre audio musical [13]. Su propuesta combina objetivos acústicos y de alto nivel durante el preentrenamiento para capturar información reutilizable. Los vectores resultantes se evaluaron en problemas de recuperación y comprensión de información musical, lo que sitúa a MERT como un encoder especializado en este dominio.

2.5. Segmentación y evaluación a nivel de canción

La segmentación aparece de forma natural en detección musical porque las canciones pueden tener duraciones superiores a las que muchos modelos procesan directamente. Trabajar con fragmentos permite adaptar el audio a modelos con entradas de duración fija, acotar el coste de cada inferencia y obtener varias observaciones de una misma pieza. La decisión a nivel de canción requiere, además, una estrategia de agregación adecuada.

Segment Transformer divide el audio en segmentos, extrae información de cada uno mediante modelos preentrenados o autosupervisados y aprende relaciones entre fragmentos para mejorar la detección a escala de canción [11]. La propuesta muestra la utilidad de separar el análisis local de los fragmentos y el modelado estructural posterior, especialmente cuando el audio completo es demasiado largo para tratarse de una sola vez.

El preprint *Fusion Segment Transformer* amplía esta idea mediante una fusión guiada de información de contenido y estructura, y proporciona una evidencia directa de evaluación sobre AIME dentro de un trabajo de detección [12]. Esta línea refuerza la relevancia de los enfoques por segmentos en problemas de detección musical.

En tareas de clasificación musical, la forma de dividir el audio y agregar predicciones puede afectar a las medidas de rendimiento. Nasrullah y Zhao muestran, en clasificación de artistas, que la longitud de los fragmentos, el esquema de particionado y la agregación a nivel de canción influyen en la evaluación [15]. Aunque la tarea es

distinta, el resultado es relevante para diseñar protocolos de segmentación que eviten la mezcla de información relacionada entre particiones.

Los trabajos específicos de SSD también refuerzan esta idea. SONICS destaca la importancia de las dependencias de larga duración, mientras que Segment Transformer modela explícitamente relaciones entre segmentos [18, 11]. Esto sugiere que un detector puede operar sobre fragmentos y, al mismo tiempo, necesitar una estrategia de agregación si se desea emitir una decisión sobre una canción completa.

2.6. Generalización, robustez y limitaciones actuales

Una dificultad recurrente en SSD es la generalización fuera del escenario de entrenamiento. Afchar et al. muestran que obtener una medida alta en una prueba cerrada resulta insuficiente para considerar resuelto el problema, ya que la robustez frente a manipulaciones del audio y la generalización a generadores no vistos siguen siendo cuestiones críticas [1]. Esta advertencia es especialmente importante cuando los detectores pueden aprender atajos acústicos asociados a un conjunto de datos, a un códec o a un proceso de producción concreto.

Los cambios de dominio también aparecen en trabajos adyacentes de SVDD. SingFake identifica dificultades con cantantes no vistos, códec de comunicación, idiomas y contextos musicales [20]. CtrSVDD, por su parte, destaca que la selección de características y la separación entre métodos de falsificación vistos y no vistos afectan al comportamiento del detector [19]. Aunque pertenecen a una tarea distinta, estos resultados muestran la fragilidad de los detectores cuando cambian las condiciones de evaluación.

Los preprints recientes de 2026 insisten en esa misma dirección desde la detección de música generada mediante IA. MusicDET plantea explícitamente la detección ante generadores no vistos mediante un enfoque *zero-shot* [9]. Broadcast Monitoring analiza un escenario donde la música aparece en fragmentos cortos o enmascarada por habla, y muestra que las condiciones de emisión pueden degradar modelos entrenados para contextos más limpios [14]. ArtifactNet, también como preprint, centra la atención en la robustez frente a códec y en detectores ligeros basados en residuos forenses [17].

En conjunto, estas aportaciones muestran que las medidas obtenidas en un corpus cerrado deben interpretarse con cautela. La duración de los fragmentos, el idioma, el artista, el género, la frecuencia de muestreo, la compresión y los artefactos de producción pueden influir en la decisión del modelo. Por tanto, una comparación experimental rigurosa debe distinguir entre rendimiento dentro del conjunto de datos evaluado y capacidad de generalización a condiciones externas.

2.7. Conclusiones

Synthetic Song Detection es una tarea reciente y todavía en desarrollo. Aunque ya existen conjuntos de datos y arquitecturas específicas para música generada de ex-

tremo a extremo, todavía falta un enfoque consolidado que pueda considerarse solución general. Los resultados en escenarios cerrados son útiles para comparar modelos, pero ofrecen garantías limitadas ante nuevos generadores, transformaciones de audio o cambios de dominio.

La revisión realizada permite justificar la comparación entre un enfoque basado en MFCC y SVM y un enfoque profundo basado en MERT congelado y SVM bajo condiciones comunes de evaluación. El primero aporta una referencia reproducible basada en características manuales, mientras que el segundo permite evaluar representaciones profundas preentrenadas orientadas al dominio musical.

Por último, AIME debe presentarse con precisión: es un corpus documentado procedente de la evaluación de modelos de generación musical, reutilizado aquí para detección binaria tras una auditoría propia. La clase IA incluye doce generadores, lo que evita depender de un único generador dentro del corpus, aunque no garantiza generalización a generadores o datasets no vistos.

3. Estudio previo

3.1. Introducción

Este capítulo presenta los objetivos del TFM, la metodología de trabajo, la planificación temporal y el presupuesto estimado del proyecto. Estos elementos establecen el marco de organización y seguimiento aplicado durante su desarrollo.

3.2. Objetivos

3.2.1. Objetivo general

Comparar un enfoque clásico basado en MFCC y SVM con un enfoque profundo basado en MERT congelado y SVM para la detección de música generada mediante inteligencia artificial, evaluando ambos mediante medidas predictivas y operacionales bajo un diseño experimental común.

3.2.2. Objetivos específicos

1. Revisar el estado del arte relacionado con la detección de música generada mediante IA.
2. Auditar y preparar AIME para su uso experimental.
3. Definir un protocolo reproducible de particionado, preprocesamiento y evaluación.
4. Seleccionar, implementar y evaluar un baseline clásico basado en MFCC y SVM.
5. Seleccionar, adaptar y evaluar un enfoque profundo basado en MERT y SVM.
6. Comparar ambos enfoques y seleccionar el modelo más adecuado para la prueba de concepto, considerando conjuntamente su rendimiento predictivo y su viabilidad operacional.
7. Desarrollar y desplegar una prueba de concepto web que integre el modelo seleccionado.
8. Documentar decisiones, experimentos y artefactos para garantizar la reproducibilidad.

3.3. Metodología

El proyecto sigue un desarrollo iterativo e incremental organizado mediante fases, issues y ciclos de trabajo breves. Esta forma de trabajo permite dividir el TFM en tareas manejables, revisar los resultados de cada etapa y mantener trazabilidad entre decisiones, experimentos y artefactos generados.

3.3.1. Documentación y revisión bibliográfica

La memoria se redacta en LaTeX mediante Overleaf, manteniendo una estructura de capítulos coherente con los requisitos del TFM. La revisión bibliográfica se organiza de forma estructurada y trazable, de modo que las fuentes empleadas puedan relacionarse con las decisiones metodológicas y con el estado del arte descrito en la memoria.

3.3.2. Gestión del proyecto y control de versiones

El desarrollo se gestiona con Git y GitHub, utilizando ramas e issues para separar tareas y registrar cada bloque de trabajo. Las decisiones relevantes, los experimentos ejecutados y los artefactos producidos se documentan en el repositorio para facilitar su revisión posterior.

3.3.3. Preparación de los datos y reproducibilidad

La preparación experimental parte de la selección y auditoría de AIME como conjunto de datos principal. Antes del procesamiento de audio se generan particiones reproducibles. El manifiesto se divide en *train*, *val* y *test*, correspondientes respectivamente al ajuste del modelo, la selección de la configuración y la evaluación final. Esta separación permite controlar fugas de información y mantener el mismo conjunto de datos para evaluar los dos enfoques comparados.

3.3.4. Desarrollo y evaluación experimental

Los modelos se evaluaron bajo el procedimiento definido durante la preparación de los datos. Los experimentos se ejecutaron en el entorno virtual del proyecto, configurado con Python 3.11.9. Durante el proceso se registraron medidas predictivas y operacionales, y la reproducibilidad se apoyó en artefactos estructurados y pruebas automatizadas sobre los componentes críticos.

La metodología experimental distingue entre cuatro formas de abordar la clasificación: características manuales con aprendizaje automático clásico, embeddings de un modelo profundo congelado con un clasificador supervisado, fine-tuning parcial y

fine-tuning completo. En el desarrollo realizado se ejecutaron las dos primeras. El baseline utilizó MFCC con una SVM, mientras que el enfoque profundo empleó MERT como encoder congelado para obtener embeddings y entrenó después una SVM sobre esas representaciones.

3.4. Planificación

La planificación del TFM se ha definido para una dedicación total de 300 horas, correspondiente a los 12 ECTS del trabajo, durante el periodo comprendido entre el 22 de junio y el 15 de septiembre de 2026. La organización sigue un enfoque iterativo e incremental: GitHub Projects y el tablero Kanban estructuran el trabajo, las tareas se registran mediante issues, los cambios se desarrollan en ramas, los commits documentan la evolución del repositorio y las pull requests facilitan la revisión e integración de cada incremento. La documentación se actualiza de forma progresiva conforme se consolidan los avances experimentales y de desarrollo.

La Tabla [3.1](#) resume las fases previstas, su periodo, la dedicación asignada y el resultado verificable asociado a cada una de ellas.

Fase	Periodo	Horas	Resultado o entregable
Alcance, planificación y entorno	22-30 de junio	20 h	Alcance y objetivos definidos, viabilidad estudiada y entorno de trabajo configurado
Auditoría de AIME y preparación de particiones	1-11 de julio	24 h	Conjunto de datos auditado y particiones reproducibles preparadas
Revisión bibliográfica y estado del arte	22 de junio-16 de julio	26 h	Estado del arte actualizado y bibliografía relevante organizada
Baseline MFCC + SVM	12-14 de julio	22 h	Baseline implementado y evaluado mediante el procedimiento experimental definido
Documentación del primer incremento	15-20 de julio	18 h	Metodología, dataset, particiones y baseline incorporados a la documentación
Selección y adaptación del modelo profundo	21 de julio-5 de agosto	52 h	Modelo profundo seleccionado, adaptado y evaluado
Comparación experimental y selección del modelo	6-12 de agosto	28 h	Comparación predictiva y operacional completada y modelo de despliegue seleccionado
Documentación del segundo incremento	13-16 de agosto	14 h	Modelo profundo, comparación, resultados y limitaciones documentados
Diseño e implementación de la aplicación web	17 de agosto-2 de septiembre	48 h	Aplicación web funcional con integración del modelo seleccionado
Pruebas, despliegue y documentación de la aplicación	3-7 de septiembre	18 h	Evidencias de prueba, despliegue académico y documentación de la aplicación
Revisión final de memoria y artefactos	8-13 de septiembre	20 h	Memoria revisada y artefactos técnicos organizados para la entrega
Preparación de la presentación	14-15 de septiembre	10 h	Presentación y demostración preparadas
Total	22 de junio-15 de septiembre	300 h	TFM preparado para su entrega

Tabla 3.1: Planificación temporal y distribución de horas del TFM.

La Figura 3.1 representa visualmente el cronograma del trabajo y permite observar el solapamiento parcial entre la revisión bibliográfica, la preparación del dataset y los primeros incrementos experimentales.

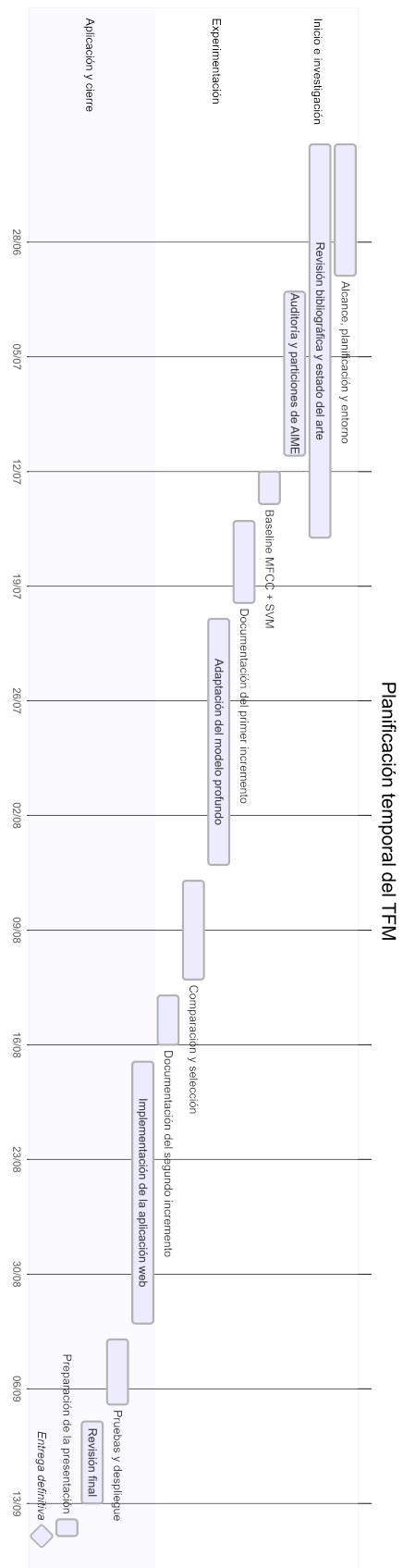


Figura 3.1: Planificación temporal del TFM entre el 22 de junio y el 15 de septiembre de 2026.

Las principales dependencias entre las fases son las siguientes:

- La auditoría de AIME y la generación de las particiones preceden a la ejecución de los experimentos.
- El baseline y el modelo profundo deben evaluarse mediante el mismo procedimiento antes de realizar la comparación.
- La selección del modelo para la aplicación depende de los resultados predictivos y operacionales de ambos enfoques.
- La implementación web requiere el artefacto y el preprocesamiento del modelo seleccionado, mientras que las pruebas y el despliegue preceden al cierre del proyecto.

3.5. Presupuesto

El presupuesto estima el coste económico asociado al desarrollo del TFM durante las 300 horas de dedicación planificada. Su finalidad es valorar el esfuerzo y los recursos necesarios para el trabajo académico, por lo que distingue entre costes imputados, como la dedicación, la amortización del hardware, la electricidad y la conexión a Internet, y costes directos de herramientas, servicios y datos. Se interpreta como una estimación presupuestaria del proyecto académico.

Recursos humanos

Para valorar la dedicación se toma como referencia el coste laboral medio por hora efectiva publicado por el Instituto Nacional de Estadística para el primer trimestre de 2026, de 24,88 €/h [10]. Aplicado a las 300 horas planificadas, el coste imputado de los recursos humanos asciende a 7.464,00 €.

$$300 \text{ h} \times 24,88 \text{ €/h} = 7.464,00 \text{ €}$$

Recursos materiales, suministros y comunicaciones

La amortización del hardware se calcula a partir de precios estimados representativos y de una vida útil de cuatro años. Esta vida útil equivale a una amortización lineal anual del 25 %, porcentaje que coincide con el coeficiente lineal máximo indicado por la Agencia Tributaria para equipos destinados a procesos de información [2]. El coste imputado al TFM se obtiene aplicando el periodo de 86 días mediante la expresión:

$$\text{coste amortizado} = \text{precio estimado} \times \frac{86}{4 \times 365}.$$

Recurso	Precio estimado	Vida útil	Periodo imputado	Coste amortizado
Portátil	900,00 €	4 años	86 días	53,01 €
Monitor	110,00 €	4 años	86 días	6,48 €
Periféricos	45,00 €	4 años	86 días	2,65 €
Total				62,14 €

Tabla 3.2: Coste imputado por amortización del hardware durante el periodo del TFM.

El coste eléctrico se estima con una potencia media de 0,12 kW para el conjunto formado por el portátil, el monitor y los periféricos. Se trata de un supuesto de consumo medio, no de una medición directa. Como precio de referencia se utiliza 0,2669 €/kWh, correspondiente a Eurostat para España durante el segundo semestre de 2025, con impuestos incluidos y banda doméstica de 2.500 a 4.999 kWh [6]:

$$0,12 \text{ kW} \times 300 \text{ h} \times 0,2669 \text{ €/kWh} = 9,61 \text{ €}$$

La conexión a Internet se valora mediante una cuota doméstica representativa de 30,00 € mensuales. Para el periodo imputado de 86 días, equivalente a 2,827 meses, el cálculo se realiza de forma proporcional sobre el año completo:

$$30,00 \times \frac{86}{365} \times 12 = 84,82 \text{ €}$$

Software, herramientas, servicios y datos

Los servicios empleados o previstos para la gestión, documentación, publicación y despliegue están disponibles mediante planes gratuitos o acceso abierto, por lo que su coste directo imputado al proyecto es de 0,00 €. La Tabla 3.3 agrupa los recursos considerados.

Recurso	Modalidad	Coste
GitHub y GitHub Projects	Plan gratuito	0,00 €
Clockify	Plan gratuito	0,00 €
Overleaf	Plan gratuito	0,00 €
Hugging Face Hub y Hugging Face Spaces	Plan gratuito	0,00 €
Vercel	Plan gratuito	0,00 €
Dataset AIME	Acceso gratuito; licencias según procedencia	0,00 €
Total de software, servicios y datos		0,00 €

Tabla 3.3: Software, herramientas, servicios y datos considerados en el presupuesto.

Resumen final

La Tabla 3.4 resume el presupuesto estimado del TFM. El coste total imputado asciende a 7.620,57 €.

Categoría	Coste
Recursos humanos	7.464,00 €
Amortización de hardware	62,14 €
Electricidad	9,61 €
Internet	84,82 €
Software, servicios y datos	0,00 €
Total estimado	7.620,57 €

Tabla 3.4: Resumen final del presupuesto estimado del TFM.

4. Descripción de la propuesta

4.1. Introducción

Este capítulo presenta el diseño experimental del TFM. A partir de AIME se construyen el subconjunto de trabajo y sus particiones, se define el baseline MFCC + SVM y se describe la adaptación de MERT como modelo profundo preentrenado para obtener embeddings y entrenar una SVM.

4.2. Formulación del problema

El problema se formula como una clasificación binaria entre música producida por humanos y música generada mediante inteligencia artificial. Cada ejemplo del subconjunto experimental recibe una etiqueta: la clase 0 corresponde a audio humano procedente de MTG-Jamendo, mientras que la clase 1 corresponde a audio generado mediante IA.

Los dos enfoques comparados trabajan sobre el mismo conjunto de ejemplos y las mismas particiones. Esta decisión permite evaluar las diferencias entre modelos sin introducir variaciones en la composición de *train*, *val* y *test*. El objetivo del experimento es estimar la capacidad de discriminación entre ambas clases.

4.3. Selección del conjunto de datos AIME

AIME se utiliza como dataset principal del TFM. La tarjeta oficial lo presenta como *AI Music Evaluation Dataset* e incluye música generada por distintos modelos, ejemplos procedentes de MTG-Jamendo y metadatos sobre su procedencia y el campo *description* [5]. Estas características permiten construir un subconjunto equilibrado y definir un protocolo propio para la clasificación binaria.

Las condiciones de uso dependen de la procedencia del contenido: las 6 000 pistas generadas se distribuyen bajo licencia CC BY 4.0; las 500 pistas procedentes de MTG-Jamendo conservan la licencia individual de cada pista; y sus etiquetas descriptivas se distribuyen bajo licencia CC BY-NC-SA 4.0 [5].

La elección de AIME se justifica por tres motivos principales:

- disponibilidad documentada de ejemplos humanos y generados;
- diversidad de generadores dentro de la clase IA;
- existencia de metadatos útiles para el etiquetado y el particionado.

Esta elección hace necesario definir un subconjunto experimental propio y un particionado adaptado a la tarea binaria del TFM.

4.4. Auditoría y construcción del subconjunto experimental

La auditoría técnica de AIME abarca 6 500 filas. En la estructura observada aparecen 500 ejemplos humanos procedentes de MTG-Jamendo y 6 000 ejemplos generados mediante IA. Estos últimos están repartidos entre doce generadores, con 500 ejemplos por generador.

En AIME, el campo *description* contiene tres etiquetas separadas por comas que describen cada ejemplo [5]. En los metadatos auditados se identificaron 500 valores distintos; por ejemplo, *ambient*, *blues*, *piano*. Cada uno aparece en trece filas: una asociada a un audio humano de MTG-Jamendo y doce a versiones generadas, una por cada modelo incluido en el conjunto de datos. Los trece registros asociados a un mismo valor de *description* se consideran un grupo semántico.

Para construir el subconjunto experimental se conserva el audio humano y se selecciona una de las versiones generadas de cada grupo. Así se obtiene un total de 1 000 ejemplos, distribuidos de forma equilibrada entre 500 humanos y 500 generados mediante IA. Los 500 ejemplos generados seleccionados se reparten de forma prácticamente uniforme entre los doce modelos, con el objetivo de mantener la diversidad dentro de la clase IA.

La regla de etiquetado binario se define a partir del campo *model*. Las muestras cuyo valor es MTG-Jamendo se etiquetan como clase humana, mientras que las asociadas al resto de modelos se consideran generadas mediante IA. Esta regla es coherente con la estructura observada y permite formular el experimento como una clasificación binaria.

4.5. Diseño de las particiones

Las particiones se definen antes de extraer tramos temporales o características, de modo que todas las transformaciones posteriores respeten la asignación original de cada ejemplo. El manifiesto experimental reúne 1 000 registros distribuidos entre *train*, *val* y *test*, con balance entre las dos clases en los tres subconjuntos, como recoge la Tabla 4.1.

Partición	Total	Humanos	Generados
<i>train</i>	700	350	350
<i>val</i>	150	75	75
<i>test</i>	150	75	75

Tabla 4.1: Distribución de ejemplos por partición y clase en el manifiesto experimental.

El reparto se realiza utilizando los grupos definidos en la sección anterior. Todos los ejemplos de un mismo grupo permanecen juntos, y el audio humano y la generación seleccionada se asignan al mismo subconjunto. Esta regla reduce el riesgo de fuga semántica entre *train*, *val* y *test*.

La división se genera con la semilla 42 y mantiene una distribución equilibrada por generador. Las pruebas automáticas verifican el balance de clases, la reproducibilidad del manifiesto, la ausencia de solapamiento entre grupos y la presencia de los doce generadores en cada subconjunto.

En el subconjunto experimental están representados los doce generadores disponibles. La distribución es prácticamente uniforme: ocho generadores aportan 42 ejemplos, mientras que AudioLDM 2 Large, MusicGen Medium, Riffusion y Suno v3.5 aportan 41 ejemplos.

4.6. Baseline clásico MFCC + SVM

El baseline clásico se implementa como un pipeline de procesamiento a nivel de fragmento. Los audios se cargan con el tipo de dato float32 y, cuando contienen varios canales, se convierten a mono mediante el promedio de sus canales.

Para cada audio se examinan los posibles tramos consecutivos de diez segundos. La energía de cada uno se calcula mediante la suma de los cuadrados de sus amplitudes, y se conserva la ventana de máxima energía, entendida como la de mayor actividad acústica. Los audios de menos de diez segundos se completan con ceros hasta alcanzar la duración objetivo. El clip seleccionado se remuestrea a 16 kHz, por lo que los diez segundos de audio quedan representados por 160 000 muestras.

La representación acústica se basa en 20 MFCC calculados a partir de ese clip. Cada coeficiente varía a lo largo del tiempo, por lo que se resume mediante su media y su desviación típica. Los 20 valores medios y los 20 valores de desviación producen un vector final de 40 características, que se almacena para poder reutilizarlo durante el entrenamiento y la evaluación.

La clasificación se realiza con un pipeline formado por *StandardScaler* y una SVM con kernel RBF [4]. *StandardScaler* centra cada característica y la escala según la desviación calculada en *train*. La SVM utiliza el kernel RBF para modelar una frontera de decisión no lineal sobre el espacio de características.

La selección de hiperparámetros contempla 16 configuraciones, resultantes de combinar $C \in \{0,1; 1; 10; 100\}$ y $\gamma \in \{scale; 0,001; 0,01; 0,1\}$. Cada configuración se entrena con *train* y se evalúa en *val*. El criterio principal es *balanced accuracy*; en caso de empate, se utiliza F1 de la clase IA como segundo criterio. La evaluación final se realiza en *test*.

4.7. Modelo profundo preentrenado MERT

El enfoque profundo se basa en m-a-p/MERT-v1-95M, empleado como encoder musical para obtener representaciones de dimensión fija. El encoder se mantiene congelado durante la extracción y los embeddings resultantes se utilizan para entrenar un clasificador SVM.

La elección de MERT se justifica por su orientación al dominio musical, la posibilidad de reutilizar embeddings de dimensión fija y su encaje con una estrategia de precomputación de características. Esta decisión también responde a las restricciones del entorno de trabajo, con 16 GB de RAM y una GPU disponible de 6 GB de VRAM. La viabilidad en CPU se comprobó mediante un smoke test previo, antes de extraer embeddings para el subconjunto completo.

Cada ejemplo se procesa como un clip de diez segundos obtenido mediante el criterio temporal definido para el baseline y se remuestrea a 24 kHz. El pipeline lo divide en dos ventanas contiguas de cinco segundos, que se procesan por separado mediante MERT. Para cada ventana se obtiene el último hidden state, es decir, la secuencia de representaciones producida por la última capa del encoder. Sobre esta secuencia se aplica un pooling temporal mediante la media, obteniendo un único vector por ventana. Finalmente, los vectores de ambas ventanas se promedian para producir un embedding de 768 componentes por clip. El conjunto completo se almacena como una matriz float32 de forma (1000, 768).

Sobre estos embeddings se entrena un clasificador formado por *StandardScaler* y una SVM. La selección considera candidatos con kernel lineal y RBF. Todos los candidatos se ajustan con *train* y se seleccionan con *val*. La regla de selección maximiza *balanced accuracy*; en caso de empate, considera ROC-AUC, F1 de la clase IA, menor latencia de predicción y preferencia por el kernel lineal. La configuración seleccionada es una SVM lineal con $C = 0,1$.

4.8. Medidas de evaluación

La evaluación predictiva emplea la clase IA como clase positiva. Para definir las medidas utilizadas se adopta la siguiente notación:

- **Verdaderos positivos (TP):** ejemplos de la clase IA clasificados correctamente como IA.
- **Verdaderos negativos (TN):** ejemplos humanos clasificados correctamente como humanos.
- **Falsos positivos (FP):** ejemplos humanos clasificados incorrectamente como IA.
- **Falsos negativos (FN):** ejemplos de la clase IA clasificados incorrectamente como humanos.

La medida principal utilizada para seleccionar las configuraciones es *balanced accuracy*, ya que promedia el recall de ambas clases y asigna el mismo peso a la clase humana y a la clase IA:

$$\text{balanced accuracy} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right).$$

Para la clase IA se emplean además las siguientes medidas:

$$\begin{aligned} \text{precision} &= \frac{TP}{TP + FP}, & \text{recall} &= \frac{TP}{TP + FN}, \\ F1 &= 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}. \end{aligned}$$

También se registran precision IA, recall IA y F1 IA para describir el comportamiento sobre la clase positiva. ROC-AUC se utiliza como medida complementaria para valorar la capacidad del modelo de ordenar correctamente los ejemplos de ambas clases a partir de los *scores* continuos de la SVM, sin depender de un único umbral de decisión [7]. La matriz de confusión permite desglosar los resultados por clase e identificar el número de falsos positivos y falsos negativos.

Para valorar la viabilidad de integración y ejecución de los enfoques en la prueba de concepto web se consideran dos medidas comparables entre los clasificadores, una medida específica del enfoque profundo y un criterio cualitativo de integración:

- **Latencia del clasificador:** se mide sobre las representaciones previamente calculadas. En MFCC + SVM se aplica sobre características MFCC ya extraídas, mientras que en MERT + SVM se aplica sobre embeddings ya obtenidos.
- **Tamaño del clasificador serializado:** representa el espacio ocupado por el artefacto formado por *StandardScaler* y SVM en cada enfoque. En el caso de MERT no incluye el processor ni el encoder.
- **Tiempo de inferencia del encoder MERT en CPU:** cuantifica el tiempo necesario para producir los embeddings de cada clip y caracteriza el coste temporal de la etapa de representación profunda.
- **Complejidad de integración:** considera los componentes y dependencias necesarios para ejecutar cada pipeline. MFCC + SVM requiere la extracción de características, el escalado y el clasificador, mientras que MERT + SVM incorpora el processor, el encoder, el escalado, el clasificador y las dependencias de aprendizaje profundo.

4.9. Limitaciones del diseño experimental

El subconjunto experimental contiene 1 000 ejemplos, por lo que solo cubre una parte del material disponible en AIME. Esta reducción permite trabajar con un diseño

controlado y equilibrado, aunque limita la variedad de ejemplos que intervienen en la evaluación.

La evaluación se realiza sobre un único corpus. En consecuencia, las medidas obtenidas describen el comportamiento de los modelos dentro de AIME y deben interpretarse dentro de ese alcance experimental.

La diferente procedencia de las dos clases introduce otra limitación. La clase humana procede de MTG-Jamendo, mientras que la clase generada reúne ejemplos producidos por varios modelos de IA. Esta configuración puede hacer que parte de la separación aprendida dependa de características de origen y no solo de propiedades generales de la generación musical.

Los metadatos de AIME no incluyen identificadores de canción, artista, álbum o grabación original. En consecuencia, el particionado puede controlar la repetición de valores de description, pero no establecer grupos basados en la identidad de una obra o de un intérprete.

También existen posibles atajos asociados a diferencias técnicas entre fuentes. La frecuencia de muestreo, el códec, la duración, el proceso de producción o el generador de origen pueden introducir señales que el modelo aprenda sin capturar propiedades generales de la música generada mediante IA. Además, los audios brutos presentan heterogeneidad técnica en duración, frecuencia de muestreo, número de canales, subtipo WAV y tamaño de archivo.

El enfoque MERT también introduce limitaciones propias. El encoder se mantiene congelado y la representación procede del último hidden state, resumido mediante medias temporales y media entre ventanas. Esta agregación resume la evolución temporal del fragmento y podría ocultar variaciones locales que no se evaluaron por separado. La unidad de evaluación es un clip de diez segundos, por lo que los resultados describen el comportamiento de los modelos a nivel de fragmento y no a nivel de canción completa.

5. Validación

5.1. Introducción

Este capítulo presenta la validación experimental de los dos enfoques implementados: el baseline clásico MFCC + SVM y el enfoque profundo basado en MERT congelado y SVM. Se recogen la ejecución de los experimentos, la selección de configuraciones, la evaluación final en *test*, la comparación predictiva y operacional, y la selección del modelo para la prueba de concepto web.

5.2. Baseline MFCC + SVM

Configuración y selección

El baseline procesó los 1 000 ejemplos del manifiesto experimental sin registrar fallos.

La búsqueda de hiperparámetros evaluó 16 configuraciones de SVM RBF, combinando $C \in \{0,1; 1; 10; 100\}$ y $\gamma \in \{scale; 0,001; 0,01; 0,1\}$. La configuración seleccionada fue $C = 10$ y $\gamma = 0,01$, ajustada con *train* y seleccionada con *val*.

Resultados

Partición	Balanced accuracy	Precision IA	Recall IA	F1 IA	ROC-AUC
<i>val</i>	0,7800	0,7625	0,8133	0,7871	0,8480
<i>test</i>	0,8333	0,8472	0,8133	0,8299	0,9086

Tabla 5.1: Medidas predictivas del baseline MFCC + SVM en *val* y *test*.

En *test*, el baseline obtiene 125 aciertos sobre 150 ejemplos. La matriz de confusión muestra que clasifica correctamente 64 de los 75 ejemplos humanos y 61 de los 75 ejemplos IA, con 11 falsos positivos y 14 falsos negativos. Esta distribución indica una dificultad ligeramente mayor para reconocer la clase IA, coherente con un recall humano de 0,8533 y un recall IA de 0,8133. La Figura 5.1 recoge esta distribución de aciertos y errores.

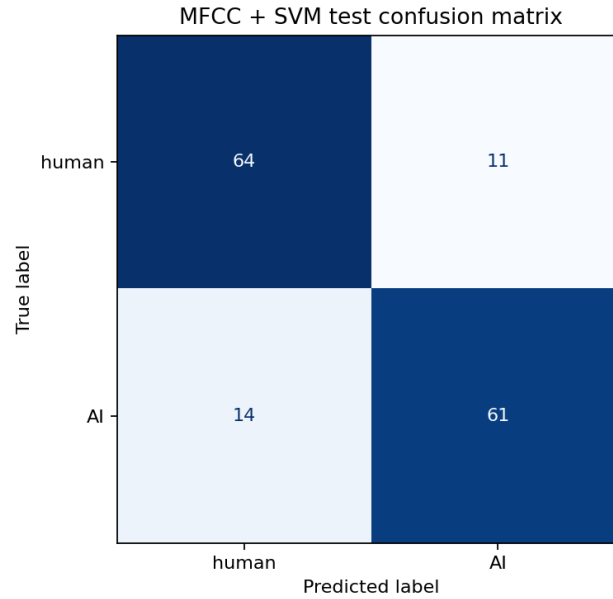


Figura 5.1: Matriz de confusión del baseline MFCC + SVM en *test*.

5.3. MERT congelado + SVM

Extracción de embeddings

Antes de procesar el subconjunto completo se ejecutó un smoke test técnico en CPU. Esta prueba verificó las dimensiones de entrada y salida, la presencia de valores finitos, el determinismo de la extracción y la viabilidad de ejecución en CPU.

La extracción completa procesó los 1 000 ejemplos del manifiesto sin fallos, IDs ausentes ni duplicados. Cada clip produjo un vector de 768 componentes y el conjunto completo quedó representado mediante una matriz de forma (1000,768). Los embeddings se almacenaron como float32 y sus metadatos se verificaron frente al manifiesto.

Selección del clasificador

El clasificador sobre embeddings se construyó como un pipeline *StandardScaler* + SVM. Se evaluaron candidatos con kernel lineal y RBF. Todos los candidatos se ajustaron con *train* y se seleccionaron con *val*. La regla de selección maximizó *balanced accuracy*; en caso de empate, consideró ROC-AUC, F1 de la clase IA, menor latencia media de predicción y preferencia por kernel lineal. La configuración seleccionada fue una SVM lineal con $C = 0,1$. Las medidas obtenidas por esta configuración en *val* se recogen en la Tabla 5.2.

Modelo	Balanced accuracy	Precision IA	Recall IA	F1 IA	ROC-AUC
MERT + SVM lineal	0,8667	0,8395	0,9067	0,8718	0,9218

Tabla 5.2: Medidas de *val* del clasificador seleccionado sobre embeddings MERT.

Resultados

Una vez fijada la configuración mediante *val*, el enfoque basado en MERT se evaluó en *test* utilizando los embeddings precomputados y una SVM lineal con $C = 0,1$. La Tabla 5.3 recoge las medidas obtenidas.

Modelo	Balanced accuracy	Precision IA	Recall IA	F1 IA	ROC-AUC
MERT + SVM lineal	0,8200	0,8158	0,8267	0,8212	0,8985

Tabla 5.3: Medidas de *test* de MERT + SVM lineal.

En *test*, el enfoque basado en MERT obtuvo 123 aciertos sobre 150 ejemplos. Clasificó correctamente 61 de los 75 ejemplos humanos y 62 de los 75 ejemplos IA, con 14 falsos positivos y 13 falsos negativos. La distribución de errores entre ambas clases fue prácticamente equilibrada, como muestra la Figura 5.2.

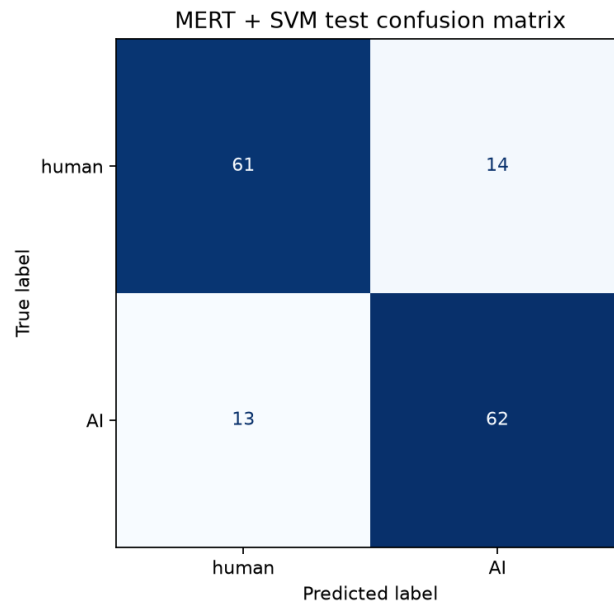


Figura 5.2: Matriz de confusión de MERT + SVM lineal en *test*.

5.4. Comparación y selección del modelo

Comparación predictiva

La Tabla 5.4 presenta las medidas de *test* de ambos enfoques y la diferencia MERT menos MFCC.

Modelo	Balanced accuracy	Precision IA	Recall IA	F1 IA	ROC-AUC
MFCC + SVM RBF	0,8333	0,8472	0,8133	0,8299	0,9086
MERT + SVM lineal	0,8200	0,8158	0,8267	0,8212	0,8985
Diferencia MERT - MFCC	-0,0133	-0,0314	+0,0133	-0,0087	-0,0101

Tabla 5.4: Comparación predictiva en *test*.

La Tabla 5.5 compara los aciertos y errores principales de ambos enfoques.

Modelo	Aciertos	Falsos positivos	Falsos negativos
MFCC + SVM RBF	125	11	14
MERT + SVM lineal	123	14	13
Diferencia MERT - MFCC	-2	+3	-1

Tabla 5.5: Comparación de aciertos, falsos positivos y falsos negativos en *test*.

En *test*, los resultados de ambos enfoques fueron próximos y deben interpretarse como comparables dentro de AIME. MFCC + SVM alcanzó una ligera ventaja en *balanced accuracy*, precision IA, F1 IA y ROC-AUC, además de obtener 125 aciertos frente a los 123 del enfoque profundo. MERT + SVM presentó un recall IA ligeramente superior y un falso negativo menos. Dado el reducido tamaño de las diferencias observadas, estos resultados no permiten afirmar una superioridad estadísticamente significativa de MFCC sobre MERT. Las diferencias se circunscriben al *test* de AIME empleado en el experimento.

Comparación operacional

La comparación operacional de la Tabla 5.6 se refiere al clasificador aplicado sobre representaciones ya calculadas. En el baseline se usan características MFCC ya extraídas, mientras que en el enfoque profundo se usan embeddings MERT ya extraídos. El tamaño corresponde al clasificador serializado; en el caso de MERT no incluye el processor ni el encoder.

Modelo	Latencia (ms/ejemplo)	Tamaño (MB)
MFCC + SVM RBF	0,090	0,14
MERT + SVM lineal	0,113	1,69

Tabla 5.6: Comparación operacional de los clasificadores.

Las latencias de 0,090 y 0,113 ms por ejemplo proceden de una única temporización sobre los 150 ejemplos de *test*; en ambos casos, la latencia se obtuvo dividiendo el tiempo total de la predicción y de la obtención de los *decision scores* entre 150. Estas mediciones cubren únicamente los clasificadores sobre representaciones previamente calculadas: las 40 características MFCC y los embeddings MERT de 768 dimensiones, respectivamente. No se registraron repeticiones independientes ni un *warm-up*, por lo que se utilizan solo como referencia operacional y no como un benchmark riguroso. Por su parte, el tiempo aproximado de 0,824 s por clip de MERT corresponde a la media de los pases del encoder sobre los 1 000 clips procesados en CPU, con tamaño de lote 1 y dos ventanas de cinco segundos por clip; no incluye decodificación, preparación con el *processor*, *pooling* ni escritura, por lo que tampoco constituye una medición extremo a extremo.

La diferencia entre las latencias de los clasificadores es pequeña y no debe interpretarse como una ventaja temporal fuerte entre los pipelines completos. La ejecución de MERT requiere además el *processor*, el encoder profundo y las dependencias de aprendizaje profundo, mientras que los 1,69 MB informados corresponden solo a *StandardScaler* + SVM y no al pipeline MERT completo. Por tanto, la evidencia operacional disponible es parcial: el clasificador serializado de MFCC ocupa menos espacio y su pipeline requiere menos componentes, pero las mediciones presentadas no comparan de forma homogénea la ejecución completa de ambos enfoques.

Selección

Las configuraciones de ambos enfoques se seleccionaron con *train* y *val*, y *test* se reservó para una única evaluación final comparativa. Una vez observados esos resultados, la elección del modelo integrado en la aplicación web se realizó como una decisión de ingeniería, sin reajustar hiperparámetros utilizando *test*.

MFCC + *StandardScaler* + SVM RBF fue seleccionado como modelo para VeriSon. Los dos enfoques presentan un rendimiento similar dentro del *test* de AIME, por lo que esta decisión no implica haber demostrado una superioridad científica concluyente de MFCC. Para el MVP, MFCC ofrece una combinación más favorable desde el punto de vista de la integración y el despliegue: su pipeline es más simple, no requiere ejecutar un encoder profundo durante la inferencia, incorpora menos dependencias y presenta una menor complejidad operacional. Su clasificador serializado también es más pequeño; en cambio, el tamaño informado para MERT corresponde solo a *StandardScaler* + SVM y no incluye el encoder ni las dependencias adicionales necesarias para el pipeline profundo completo.

6. Aplicación web VeriSon

6.1. Objetivo y alcance

Como prueba de concepto, se ha desarrollado VeriSon, una aplicación web que permite analizar canciones y presentar una estimación sobre el posible origen del contenido, distinguiendo entre origen humano y generación mediante IA. La aplicación integra el enfoque seleccionado en el capítulo 5: MFCC + *StandardScaler* + SVM RBF.

Su funcionamiento es el siguiente: el usuario proporciona un archivo de audio, que se somete a las etapas de preprocesamiento, extracción de características y clasificación descritas en este capítulo. Tras el análisis, la interfaz presenta una estimación cualitativa sobre su posible origen. De este modo, la aplicación conecta el trabajo experimental con una solución software utilizable.

6.2. Arquitectura y tecnologías

VeriSon adopta una arquitectura cliente-servidor formada por dos componentes independientes: una aplicación web de una sola página que organiza la interacción con el usuario y un servicio backend que encapsula la API y el pipeline de inferencia. El archivo de audio se envía al backend mediante una petición HTTP *multipart/form-data*, mientras que el resultado del análisis se devuelve en una respuesta JSON. Esta separación desacopla la interfaz de la lógica de análisis y permite desplegar ambos componentes de forma independiente.

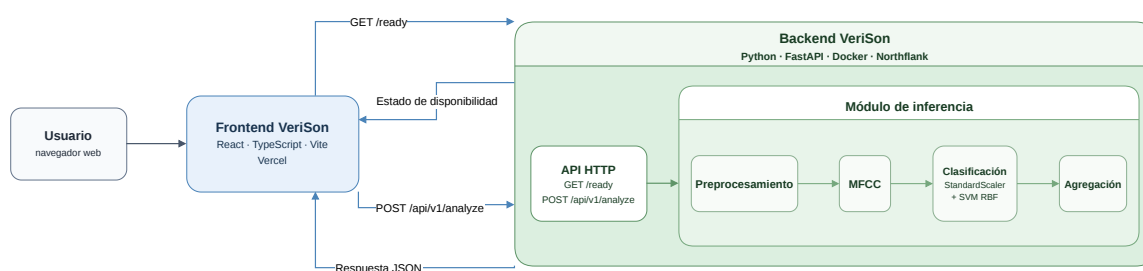


Figura 6.1: Arquitectura general de la aplicación web VeriSon.

El frontend se implementa con React, TypeScript y Vite. React organiza la interfaz y los estados de interacción; TypeScript aporta tipado estático al cliente y a las estructuras utilizadas en la comunicación con la API, y Vite proporciona el entorno de desarrollo y construcción. El frontend se despliega en Vercel, elegido por la facilidad de despliegue de aplicaciones construidas con Vite y por disponer de una opción sin coste adecuada para las necesidades de este componente.

Dado que el pipeline de procesamiento y el modelo seleccionado se ejecutan en Python, el backend se implementa en el mismo lenguaje, lo que permite integrar directamente la lógica de inferencia. FastAPI se utiliza como framework para exponer esta funcionalidad mediante una API HTTP y Docker permite definir y reproducir el entorno del backend junto con sus dependencias. Para el despliegue del servicio contenerizado se seleccionó Northflank, al ofrecer dentro del plan utilizado recursos suficientes para ejecutar el backend sin coste para el proyecto.

El código de la aplicación web se mantiene en el mismo repositorio GitHub que el resto del proyecto y la experimentación, lo que centraliza el versionado y su evolución conjunta. Por este motivo, se utiliza GitHub Actions para la integración continua, aprovechando su integración directa con el repositorio, las *pull requests* y los *checks*.

6.3. Backend y flujo de inferencia

6.3.1. API y validación de entrada

Al iniciar el servicio, la función de ciclo de vida crea una única instancia del servicio de inferencia, carga el artefacto del modelo y verifica su compatibilidad con la configuración esperada. Si estas operaciones concluyen correctamente, el modelo queda disponible para las solicitudes posteriores; si fallan, el servicio conserva un estado degradado y comunica que el análisis no está disponible. Este comportamiento evita cargar el modelo para cada petición y permite diferenciar la disponibilidad de la API de la disponibilidad efectiva de la inferencia.

La tabla 6.1 resume los puntos de acceso públicos de la API.

Tabla 6.1: Puntos de acceso principales de la API de VeriSon.

Ruta	Método	Finalidad
/health	GET	Informa del estado general del servicio y de la carga del modelo.
/ready	GET	Indica si el servicio está preparado para aceptar análisis.
/api/v1/model	GET	Devuelve la información disponible sobre el artefacto del modelo cargado.
/api/v1/analyze	POST	Recibe un archivo de audio y devuelve el resultado de la inferencia.

Por su parte, *GET /ready* comprueba específicamente si el servicio está preparado para aceptar análisis. Durante la inicialización o el *warm-up*, la API puede ser accesible y responder a la comprobación general, pero este endpoint mantiene el estado de no disponibilidad para análisis hasta que finaliza dicha preparación.

La solicitud *POST /api/v1/analyze* recibe el archivo mediante *multipart/form-data*. Se aceptan archivos WAV y MP3 y, antes de iniciar la extracción de características y la clasificación, el backend realiza las siguientes validaciones:

- Extensión y, cuando está disponible, tipo MIME.
- Contenido no vacío.
- Tamaño máximo, configurable y de 64 MiB por defecto.
- Duración máxima, configurable y de 300 s por defecto.
- Posibilidad de decodificar correctamente el audio.

Cuando alguna validación falla, la API devuelve una respuesta controlada y diferenciada según la causa. También existen respuestas controladas cuando el modelo no está disponible o se produce un fallo durante la inferencia, lo que permite al frontend traducirlas a mensajes adecuados.

Para procesar la carga, el backend copia el contenido recibido en un archivo temporal. Dicho archivo se utiliza durante la decodificación y se elimina tanto tras una inferencia correcta como cuando se produce una excepción, de modo que no se acumulan archivos de usuario en el servicio.

6.3.2. Preprocesamiento e inferencia

El archivo de audio se recorre de forma secuencial en fragmentos de hasta 10 s. Cada bloque se convierte a mono cuando procede y se remuestrea a 16 kHz; únicamente el último se completa mediante *padding* cuando no alcanza la duración objetivo. Para cada fragmento se conserva además su duración original, utilizada posteriormente en la agregación de las decisiones parciales.

El tratamiento temporal difiere del empleado en el baseline experimental. Cuando un audio superaba los 10 s, el baseline seleccionaba una única ventana de 10 s de máxima energía. VeriSon, en cambio, analiza todos los bloques obtenidos mediante la segmentación anterior para cubrir el archivo proporcionado por el usuario.

Para cada fragmento se calculan 20 coeficientes MFCC. La media y la desviación típica temporal de cada coeficiente forman un vector de 40 características, que se transforma con el *StandardScaler* y se clasifica mediante la SVM con núcleo RBF. El clasificador produce una puntuación de decisión, cuyo signo se compara con el umbral global cero.

6.3.3. Agregación y respuesta

Para obtener una única estimación a nivel de archivo, las puntuaciones de los fragmentos se combinan mediante una media ponderada por su duración original. Si s_i representa la puntuación del fragmento i y d_i su duración antes de aplicar el *padding*, de forma que el relleno del último fragmento no incremente artificialmente su peso, la puntuación global se calcula como:

$$S = \frac{\sum_{i=1}^n d_i s_i}{\sum_{i=1}^n d_i}.$$

El signo de S se compara con el mismo umbral global cero utilizado en cada fragmento para obtener la clase final. A partir de este resultado, la respuesta JSON agrupa la siguiente información:

- el resultado global y su puntuación.
- la información del audio analizado, como la duración y el número de fragmentos.
- los resultados individuales de los fragmentos.
- los tiempos e información técnica del modelo.

Esta estructura permite al frontend mostrar la información necesaria para la interfaz, mientras que la API mantiene el detalle técnico del análisis.

6.4. Frontend e interacción con el usuario

6.4.1. Flujo de interacción

La interfaz guía al usuario desde la selección del archivo hasta la visualización del resultado. Antes de iniciar el análisis, valida localmente que la extensión corresponda a WAV o MP3. El flujo distingue los estados *idle*, *selected*, *preparing*, *analyzing*, *success* y *error*, e impide solicitudes duplicadas mientras existe una operación activa. El usuario puede volver a seleccionar otro archivo mediante el reinicio del flujo; si existe una petición activa, esta se cancela al reiniciarlo o cuando el componente se desmonta.

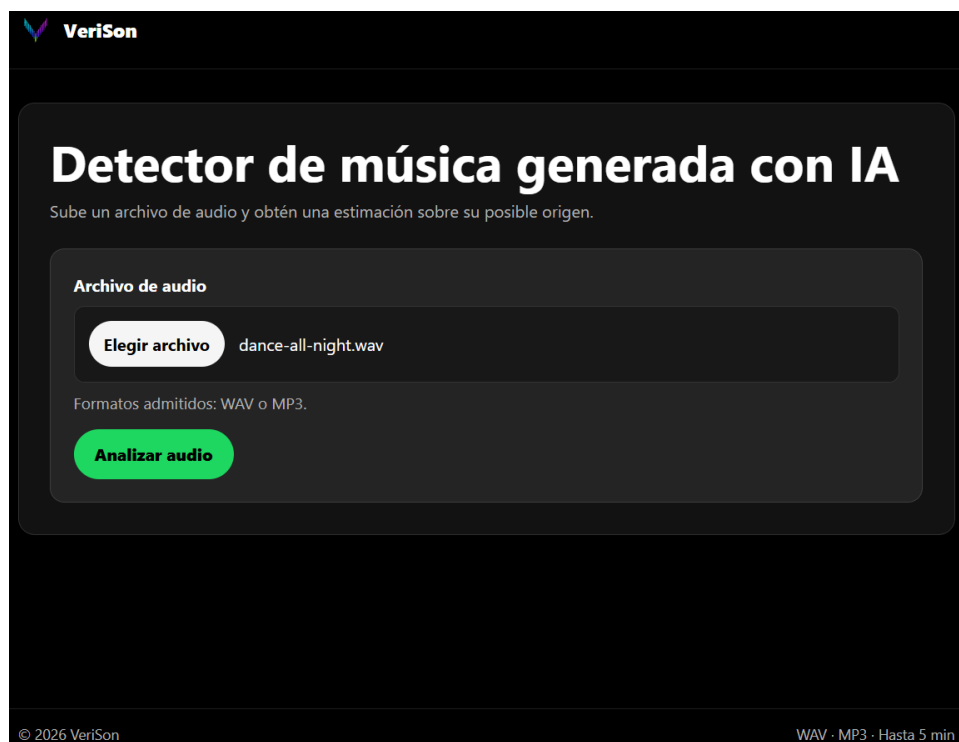


Figura 6.2: Estado de VeriSon tras seleccionar un archivo WAV válido.

6.4.2. Comunicación con el backend

La URL pública del backend se configura mediante una variable de entorno del frontend. Antes de enviar el archivo, el cliente consulta *GET /ready*. Las respuestas transitorias de disponibilidad se reintentan durante un presupuesto total de 15 s, con sondeos cada 5 s y un límite individual de 5 s por consulta. Cuando el backend informa que está preparado, el cliente envía *POST /api/v1/analyze* con el audio en *multipart/form-data*. La solicitud de análisis dispone de un *timeout* de 180 s y no se reintenta automáticamente, para evitar duplicar la operación ante una respuesta incierta.

6.4.3. Presentación del resultado

Tras una respuesta correcta, la interfaz muestra el nombre del archivo, la etiqueta *Posible generación con IA* o *Posible origen humano* y un mensaje explicativo asociado al resultado. Ante un error, interpreta la respuesta técnica del backend, la transforma en un mensaje comprensible, refleja el estado *error* y permite volver al flujo normal. También gestiona errores propios del cliente, como problemas de red o *timeout*.

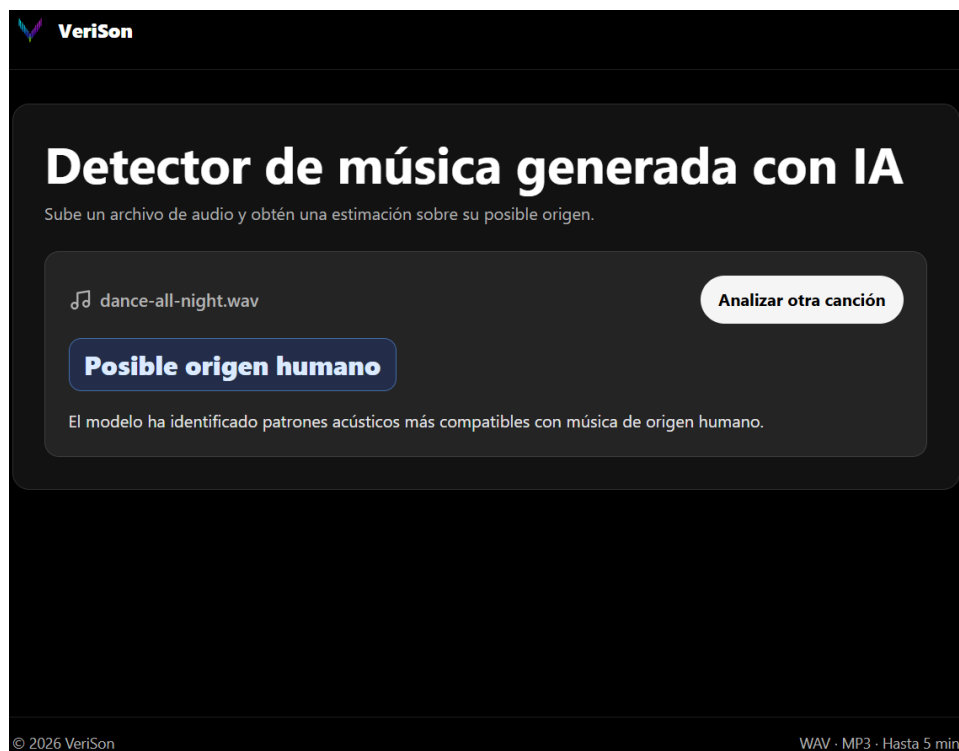


Figura 6.3: Resultado de un análisis con un archivo WAV.

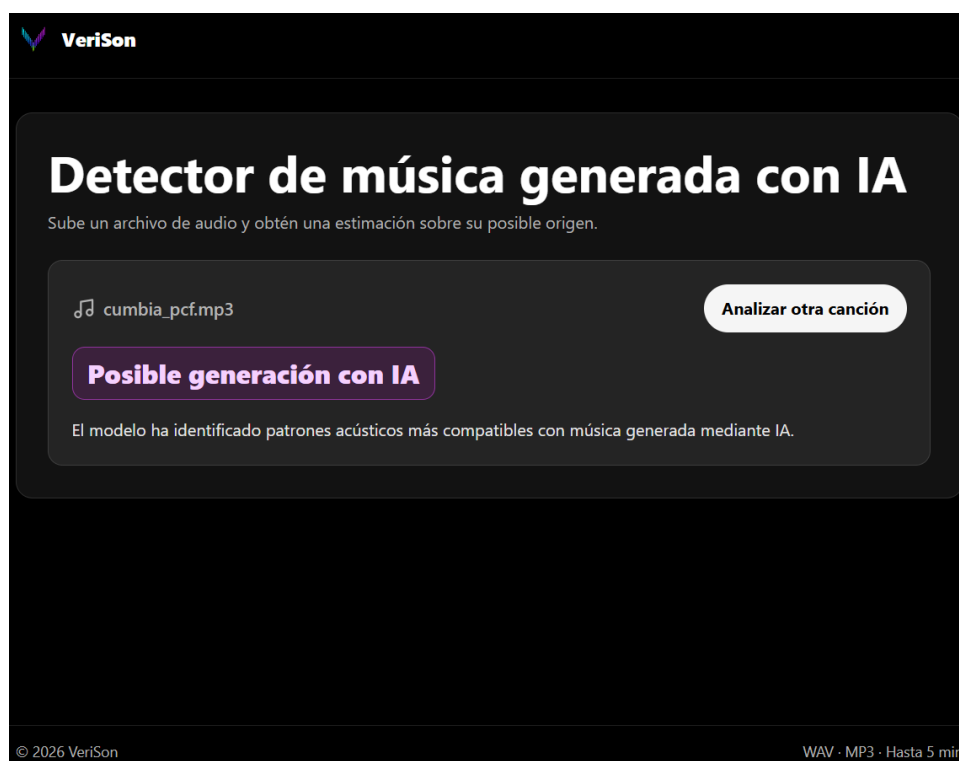


Figura 6.4: Resultado de un análisis con un archivo MP3.

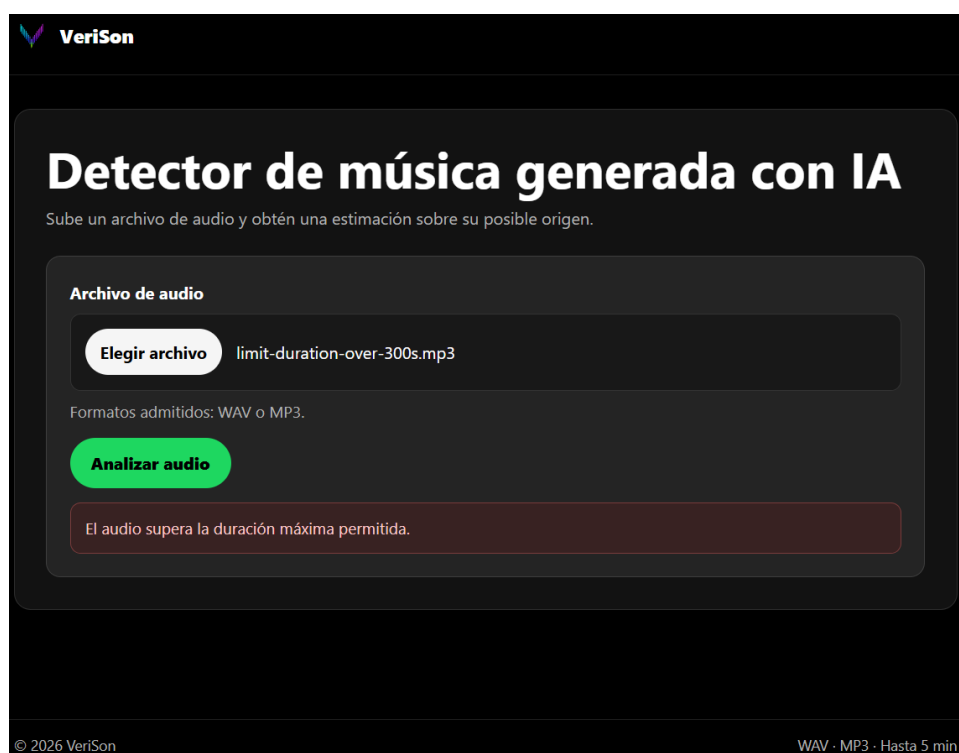


Figura 6.5: Mensaje mostrado tras rechazar un audio que supera la duración máxima permitida.

6.5. Despliegue e integración continua

6.5.1. Despliegue

El frontend se despliega en Vercel y el backend, junto con el artefacto del modelo, se ejecuta en Northflank mediante una imagen Docker. La URL pública del backend se proporciona al frontend mediante su configuración de entorno, y la política CORS del backend autoriza el origen del cliente desplegado. La imagen Docker instala las dependencias de Python, incorpora el código del servicio y el artefacto necesario para la inferencia, e inicia el servicio FastAPI mediante Uvicorn en el puerto indicado por el entorno.

El servicio desplegado en Northflank utiliza una única instancia y dispone de 0,2 vCPU compartida, 512 MiB de memoria y 1 GiB de almacenamiento efímero. Debido al límite de memoria disponible, el procesamiento del audio se reorganizó para reducir la cantidad de datos mantenidos simultáneamente. Para ello, el archivo se abre una sola vez y se recorre de forma secuencial, procesando cada fragmento y transformándolo inmediatamente en su vector de 40 características. Hasta la clasificación solo se conservan estos vectores y sus metadatos, evitando mantener en memoria al mismo tiempo el audio completo y las señales intermedias de todos los fragmentos.

También se incorporó un *warm-up* durante el arranque para evitar que el coste de inicialización recaiga sobre el primer análisis. Tras cargar el artefacto del modelo, y con la configuración de despliegue activada, el servicio ejecuta en segundo plano un remuestreo sintético de 1 s desde 48 kHz hasta 16 kHz y, si este concluye correctamente, una extracción MFCC sobre una señal sintética de la longitud objetivo. Ambas operaciones utilizan las mismas funciones que el análisis. Mientras este proceso está pendiente, *GET /ready* no declara preparado el servicio y el frontend espera antes de enviar un archivo real. Cuando las dos operaciones finalizan correctamente, el estado de preparación pasa a disponible; si alguna falla, permanece no disponible para análisis.

6.5.2. Integración continua

El workflow de GitHub Actions se ejecuta ante cada *pull request* a *main* y cada *push* a *main*. La protección de esta rama requiere una *pull request* y que los tres *checks* definidos a continuación finalicen correctamente antes del *merge*:

- **Backend:** utiliza Python 3.11.9, instala las dependencias y ejecuta la suite implementada con *pytest*, que cubre el procesamiento e inferencia, las validaciones de la API y la inicialización y disponibilidad del servicio.
- **Frontend:** utiliza Node 20.19.0, instala las dependencias y ejecuta *npm run check*, un comando del proyecto que agrupa el *lint*, la comprobación del *build* y las pruebas automatizadas con *Vitest*. Estas verifican el comportamiento de la interfaz, la validación de formatos, la comunicación con la API y la gestión de estados y errores.

- **Docker:** comprueba que la imagen del backend puede construirse correctamente mediante el Dockerfile de despliegue.

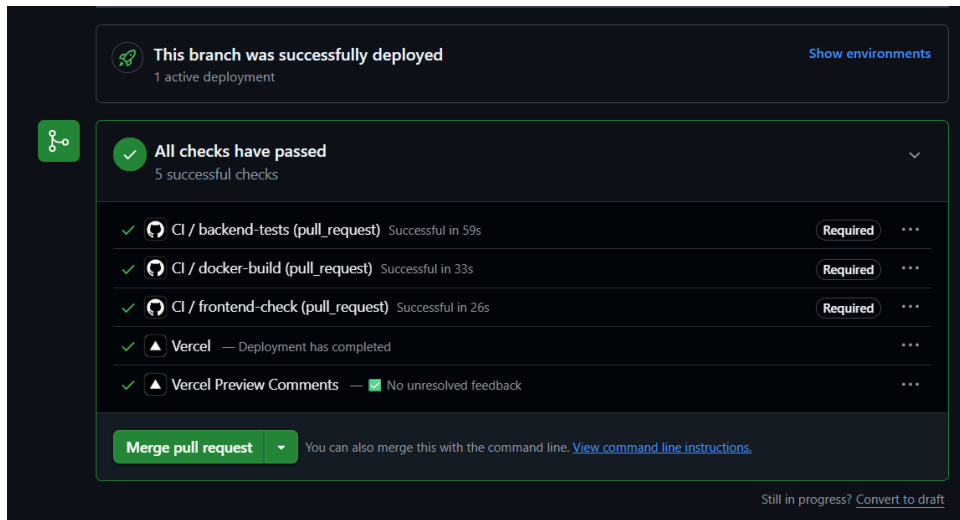


Figura 6.6: Checks obligatorios de integración continua superados en una *pull request*.

GitHub Actions se utiliza exclusivamente como mecanismo de integración continua: verifica el código, las pruebas y el empaquetado antes de integrar cambios en *main*, pero no publica la imagen Docker ni realiza los despliegues en Northflank o Vercel. Por tanto, no constituye un pipeline completo de CI/CD.

7. Conclusiones

8. Bibliografía

- [1] Darius Afchar, Gabriel Meseguer-Brocal, and Romain Hennequin. 2025. AI-Generated Music Detection and its Challenges. arXiv:2501.10111 [cs.SD]
- [2] Agencia Estatal de Administración Tributaria. 2026. Por coeficientes de amortización lineal. <https://sede.agenciatributaria.gob.es/Sede/ayuda/manuales-s-videos-folletos/manuales-practicos/irpf-2025/c07-rendimientos-actividades-economicas-estimacion-directa/fase-1-determinacion-rendimiento-neto/amortizaciones-dotaciones-ejercicio-fiscalmente-deducibles/requisitos-generales/coeficientes-amortizacion-lineal.html>. Manual práctico de Renta 2025, página actualizada el 17 de marzo de 2026, consultada el 21 de julio de 2026.
- [3] Luca Comanducci, Paolo Bestagini, and Stefano Tubaro. 2024. FakeMusicCaps: a Dataset for Detection and Attribution of Synthetic Music Generated via Text-to-Music Models. arXiv:2409.10684 [cs.SD]
- [4] Corinna Cortes and Vladimir Vapnik. 1995. Support-vector networks. *Machine Learning* 20 (1995), 273–297. doi:10.1007/BF00994018
- [5] disco-eth. s. f. AIME: AI Music Evaluation Dataset. <https://huggingface.co/datasets/disco-eth/AIME>. Dataset card de Hugging Face, consultada el 15 de julio de 2026.
- [6] Eurostat. 2026. Electricity prices for household consumers – bi-annual data (from 2007 onwards) (nrg_pc_204). https://ec.europa.eu/eurostat/databrowser/view/nrg_pc_204/default/table. doi:10.2908/NRG_PC_204 Datos actualizados el 19 de mayo de 2026; filtro: España, 2025-S2, EUR/kWh, impuestos incluidos, banda doméstica de 2.500 a 4.999 kWh; consultada el 21 de julio de 2026.
- [7] Tom Fawcett. 2006. An introduction to ROC analysis. *Pattern Recognition Letters* 27, 8 (2006), 861–874. doi:10.1016/j.patrec.2005.10.010
- [8] Florian Grötschla, Ahmet Solak, Luca A. Lanzendörfer, and Roger Wattenhofer. 2025. Benchmarking Music Generation Models and Metrics via Human Preference Studies. arXiv:2506.19085 [cs.SD]
- [9] Chaolei Han, Hongsong Wang, and Jie Gui. 2026. MusicDET: Zero-Shot AI-Generated Music Detection. arXiv:2605.18072 [cs.SD]
- [10] Instituto Nacional de Estadística. 2026. Encuesta Trimestral de Coste Laboral (ETCL). Primer trimestre 2026. <https://ine.es/dyngs/Prensa/ETCL1T26.htm>. Publicada el 16 de junio de 2026, consultada el 21 de julio de 2026.
- [11] Yumin Kim and Seonghyeon Go. 2025. Segment Transformer: AI-Generated Music Detection via Music Structural Analysis. arXiv:2509.08283 [cs.SD]

- [12] Yumin Kim and Seonghyeon Go. 2026. Fusion Segment Transformer: Bi-Directional Attention Guided Fusion Network for AI-Generated Music Detection. arXiv:2601.13647 [cs.SD] doi:10.48550/arXiv.2601.13647
- [13] Yizhi Li, Ruibin Yuan, Ge Zhang, Yinghao Ma, Xingran Chen, Hanzhi Yin, Chenghao Xiao, Chenghua Lin, Anton Ragni, Emmanouil Benetos, Norbert Gyenge, Roger Dannenberg, Ruibo Liu, Wenhui Chen, Gus Xia, Yemin Shi, Wenhao Huang, Zili Wang, Yike Guo, and Jie Fu. 2024. MERT: Acoustic Music Understanding Model with Large-Scale Self-supervised Training. In *The Twelfth International Conference on Learning Representations*. OpenReview.net, Vienna, Austria, 24 pages. <https://openreview.net/forum?id=w3YZ9MSlBu>
- [14] David Lopez-Ayala, Asier Cabello, Pablo Zinemanas, Emilio Molina, and Martin Rocamora. 2026. AI-Generated Music Detection in Broadcast Monitoring. arXiv:2602.06823 [cs.SD]
- [15] Zain Nasrullah and Yue Zhao. 2019. Music Artist Classification with Convolutional Recurrent Neural Networks. arXiv:1901.04555 [cs.SD]
- [16] Sergey Novoselov, Alexandr Kozlov, Galina Lavrentyeva, Konstantin Simonchik, and Vadim Shchemelinin. 2015. STC Anti-spoofing Systems for the ASVspoof 2015 Challenge. arXiv:1507.08074 [cs.SD]
- [17] Heewon Oh. 2026. ArtifactNet: Detecting AI-Generated Music via Forensic Residual Physics. arXiv:2604.16254 [cs.SD]
- [18] Md Awsafur Rahman, Zaber Ibn Abdul Hakim, Najibul Haque Sarker, Bishmoy Paul, and Shaikh Anowarul Fattah. 2024. SONICS: Synthetic Or Not – Identifying Counterfeit Songs. arXiv:2408.14080 [cs.SD]
- [19] Yongyi Zang, Jiatong Shi, You Zhang, Ryuichi Yamamoto, Jionghao Han, Yuxun Tang, Shengyuan Xu, Wenxiao Zhao, Jing Guo, Tomoki Toda, and Zhiyao Duan. 2024. CtrSVDD: A Benchmark Dataset and Baseline Analysis for Controlled Singing Voice Deepfake Detection. arXiv:2406.02438 [cs.SD]
- [20] Yongyi Zang, You Zhang, Mojtaba Heydari, and Zhiyao Duan. 2023. SingFake: Singing Voice Deepfake Detection. arXiv:2309.07525 [cs.SD]